

MyKavo — Master Case Study & Architecture Reference

Know what changed. Fix what matters.

A comprehensive, end-to-end technical deep-dive into the architecture, design system, database schema, scanner engine, comparison logic, severity scoring, security pipeline, background workers, and deployment guardrails of MyKavo.

1. Executive Summary & Product Vision

1.1 Product Purpose

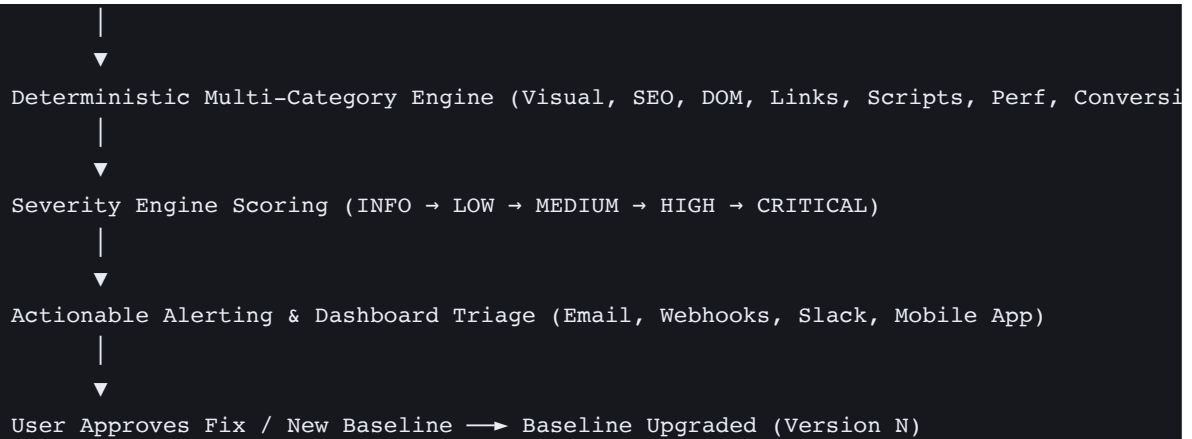
MyKavo is a professional, high-precision website change detection and regression monitoring SaaS platform. It acts as the continuous monitoring layer between website deployment and customer impact.

Modern web development, SEO, and maintenance teams manage complex digital properties across WordPress, Shopify, Webflow, and custom stacks. When a site undergoes a deployment, plugin update, CMS content edit, or third-party script change, critical elements often break silently—such as a missing CTA button, a broken lead form, an altered `robots.txt`, a dropped canonical tag, or a visual layout spill.

MyKavo solves this by creating an **approved baseline snapshot** of monitored website pages, continuously re-scanning them on automated schedules, comparing new captures against the baseline, detecting deterministic changes, scoring severity levels, displaying visual side-by-side diffs, and dispatching actionable alerts before customer revenue or SEO rankings are damaged.

1.2 The Core Product Loop

```
graph TD
    A[Target Website] --> B[Create Approved Baseline Snapshot (Version 1)]
    B --> C[Continuous Automated Scanning (Daily / Weekly Cron)]
```



1.3 Scope Boundaries: What MyKavo IS vs. What It IS NOT

To maintain high precision and low false-positive rates, MyKavo adheres to strict product boundaries:

- **MyKavo IS:** A website regression and change detection platform providing deterministic visual diffs, conversion CTA tracking, site metadata watching, script monitoring, and severity-scored alerts.
- **MyKavo IS NOT:** An Ahrefs/Semrush backlink & keyword platform, a Screaming Frog technical crawler, a generic uptime ping monitor, a heavy APM/observability agent, or a black-box "AI summary" generator.

2. Complete Technology Stack Matrix (Start to End)

The codebase is organized as a unified, type-safe TypeScript monorepo using **pnpm workspaces**.

Layer / Concern	Technology / Library	Version / Details	Role in MyKavo
Monorepo Manager	pnpm	10.18.3	Manages <code>apps/</code> and <code>packages/</code> dependencies with workspace resolution
Task Orchestration	concurrently	^9.2.1	Concurrent dev server execution (<code>pnpm dev:all</code>)
Frontend Framework	Next.js	15.1.x (App Router)	Server Components, Client Components, Route Handlers (<code>apps/web</code>)
UI Library & Logic	React	19.0.x	Modern reactive UI rendering and hooks

Layer / Concern	Technology / Library	Version / Details	Role in MyKavo
Styling Engine	Tailwind CSS	v4.0 (@import "tailwindcss")	High-performance CSS engine using inline @theme tokens
Icons & Assets	Lucide React	^0.469.0	Technical icon set
Fonts	Google Fonts / Local	Poppins, DM Sans, Geist Mono	Typography design hierarchy
Charts & Graphics	Recharts	^2.15.0	Lighthouse trend charts and availability meter visualizations
Form Management	React Hook Form + Zod	^7.54.2 / ^3.24.1	Schema-validated user inputs, profile forms, selector settings
Database ORM	Prisma	^6.2.1	Type-safe schema generator, migration engine, client library
Database Engine	PostgreSQL	15+ (Neon / Supabase / RDS)	Relational database engine with ACID transactions & indexing
Prisma Engine Targets	native , rhel-openssl-3.0.x	AWS Lambda / Netlify	Compatible with local macOS/Linux and Netlify Serverless Functions
Authentication	Better Auth	^1.1.14	Password, OAuth, Session management, TOTP Two-Factor Auth (2FA)
Browser Automation	Playwright	^1.49.1 (bundled Chromium)	Headless browser pool for rendering, DOM capture, and screenshots
Performance Audits	Lighthouse CLI	^12.3.0	On-demand CWV (LCP, FCP, TBT, TTI, CLS) performance score audits
Visual Diffing	Pixelmatch + PNGjs	^7.1.0 / ^2.3.1	Pixel-level visual regression screenshot comparison
HTML/DOM Diffing	Cheerio + Crypto	^1.0.0 / Node native	Fast server-side DOM parsing, element stripping, SHA-256 hashing

Layer / Concern	Technology / Library	Version / Details	Role in MyKavo
Background Queue	pg-boss / Custom Cron	Node.js TS	Postgres-backed queue & cron intervals for sweeps and scan dispatch
Object Storage	S3 API / Cloudflare R2	AWS SDK v3 (@aws-sdk/client-s3)	Screenshot PNG storage, signed URL generation, lifecycle retention
Email Dispatch	Resend + React Email	^4.0.1 / @react-email/	Transactional email rendering for scan alerts, invites, billing
Payments & Billing	Dodo Payments	REST API + Webhooks	Plan management (Free vs \$12/mo Pro), add-on packs (\$6/mo), checkout intents
Mobile Client	Expo (React Native)	SDK 52 / Expo Router	Cross-platform iOS/Android app for push notifications and change triage
Testing Framework	Vitest	^2.1.8	Unit, integration, theme-contrast, SSRF, and severity engine tests

3. Brand Identity & Design System

MyKavo's visual language is engineered to feel **technical, precise, trustworthy, developer-friendly, and calm**. It avoids generic startup templates, purple "AI-wrapper" gradients, glassmorphism, and unlabelled status indicators.

3.1 Typography Hierarchy

Display Headings —————> Poppins (600/700 weight, tight tracking)
 Body & Subtitles —————> DM Sans / Google Sans (400/500 weight)
 Technical Data —————> Geist Mono (URLs, Hashes, Status Codes, Diff Tokens)

- **Display Stat Numbers:** 56px / 64px , 600 weight, tight tracking (`-0.02em`).
- **Heading 1 (h1):** 40px / 48px , Poppins font.
- **Heading 2 (h2):** 30px / 38px , Poppins font.
- **Heading 3 (h3):** 22px / 30px , Poppins / DM Sans.
- **Body Text:** 15px / 24px (`font-size: 15px; line-height: 1.6;`).
- **Small Labels:** 13px / 20px , 500 weight.

- **Micro-labels** (`.label-micro`): 11px / 16px , uppercase, `letter-spacing: 0.08em` , faint ink color.

3.2 Complete Color Palette & Token Matrix

The design system is implemented in Tailwind v4 (`apps/web/src/app/globals.css`) using CSS custom properties (`--fx-`) mapped to Tailwind variables via `@theme inline` . Theme switching (`data-theme="dark"`) dynamically swaps variable values without layout reflows.

Token	Light Mode (HEX)	Dark Mode (HEX)	Usage / Intent
<code>--fx-canvas</code>	<code>#ecef4</code>	<code>#0e1015</code>	Outer page backdrop background
<code>--fx-surface</code>	<code>#f6f7fb</code>	<code>#14171d</code>	App dashboard inner container background
<code>--fx-card</code>	<code>#ffffff</code>	<code>#191d24</code>	Elevated cards, side rails, table containers
<code>--fx-ink</code>	<code>#16181d</code>	<code>#e9ecf2</code>	Primary high-contrast text
<code>--fx-ink-secondary</code>	<code>#5c6270</code>	<code>#a8b0c0</code>	Secondary description text, meta labels
<code>--fx-ink-faint</code>	<code>#9aa1b1</code>	<code>#7d8596</code>	Placeholder text, disabled items
<code>--fx-ink-inverse</code>	<code>#ffffff</code>	<code>#10131a</code>	Text rendered on dark fills / primary buttons
<code>--fx-line</code>	<code>#e4e7ee</code>	<code>#262b35</code>	Subtle dividers, hairline borders
<code>--fx-panel</code>	<code>#16181d</code>	<code>#16181d</code>	Persistent dark marketing/CTA panels and code boxes
<code>--fx-primary</code>	<code>#3556f4</code>	<code>#7c92fa</code>	Royal Blue — primary actions, links, focus rings
<code>--fx-primary-hover</code>	<code>#2a46d6</code>	<code>#93a5fb</code>	Hover state for primary buttons
<code>--fx-primary-soft</code>	<code>#e8ecfe</code>	<code>#1c2547</code>	Selected pill fills, active navigation tabs
<code>--fx-success</code>	<code>#16a34a</code>	<code>#34c979</code>	Green — healthy state, resolved changes
<code>--fx-success-soft</code>	<code>#e5f6ec</code>	<code>#122b1d</code>	Soft background for success chips

Token	Light Mode (HEX)	Dark Mode (HEX)	Usage / Intent
<code>--fx-success-strong</code>	<code>#147a3a</code>	<code>#5fd695</code>	Contrast text for success badges
<code>--fx-warning</code>	<code>#f59e0b</code>	<code>#f5a623</code>	Amber — Medium severity changes, warnings
<code>--fx-warning-soft</code>	<code>#fdf3e0</code>	<code>#2e2410</code>	Soft background for warning chips
<code>--fx-orange</code>	<code>#f97316</code>	<code>#fb8b47</code>	High severity changes, urgent alerts
<code>--fx-orange-soft</code>	<code>#feeee2</code>	<code>#33200f</code>	Soft background for orange chips
<code>--fx-critical</code>	<code>#e5484d</code>	<code>#f16a6f</code>	Red — Critical severity, site DOWN, 5xx errors
<code>--fx-critical-soft</code>	<code>#fdeaeb</code>	<code>#371b1e</code>	Soft background for critical alert banners
<code>--fx-info</code>	<code>#666c7a</code>	<code>#9aa3b2</code>	Gray — Informational changes, minor edits
<code>--fx-info-soft</code>	<code>#eef0f3</code>	<code>#232833</code>	Soft background for info badges

3.3 Reserved Accent Gradients

Gradient backgrounds are strictly capped at a maximum of **two tiles per screen** (used only for high-priority overview stat tiles):

- `--fx-gradient-coral` : `linear-gradient(135deg, #fde5d8 0%, #e9d5f2 45%, #fbc7b6 100%)` (Attention / Changes card)
- `--fx-gradient-mint` : `linear-gradient(135deg, #d8f4ee 0%, #bfeef2 50%, #8fd8ee 100%)` (Healthy / Active Monitoring card)

3.4 Shape, Elevation & Component Rules

- Card Radius:** `24px` (`rounded-3x1`), with soft dual-layer ambient shadows (`shadow-card`).
- Tile Radius:** `16px` , used for nested sub-cards.
- Input Radius:** `12px` , with explicit `2px` primary focus outline.
- Pills & Buttons:** `9999px` (`rounded-full`), pill-shaped UI controls.
- Accessibility Rule:** Severity status must NEVER be communicated by color alone. Every badge pairs a colored indicator dot with an explicit uppercase text label (e.g., `CRITICAL` , `HIGH` , `MEDIUM`). Contrast is unit-tested against WCAG AA standards (`theme-contrast.test.ts`).

4. Architecture & Monorepo Structure

Fluxen (MyKavo Monorepo)

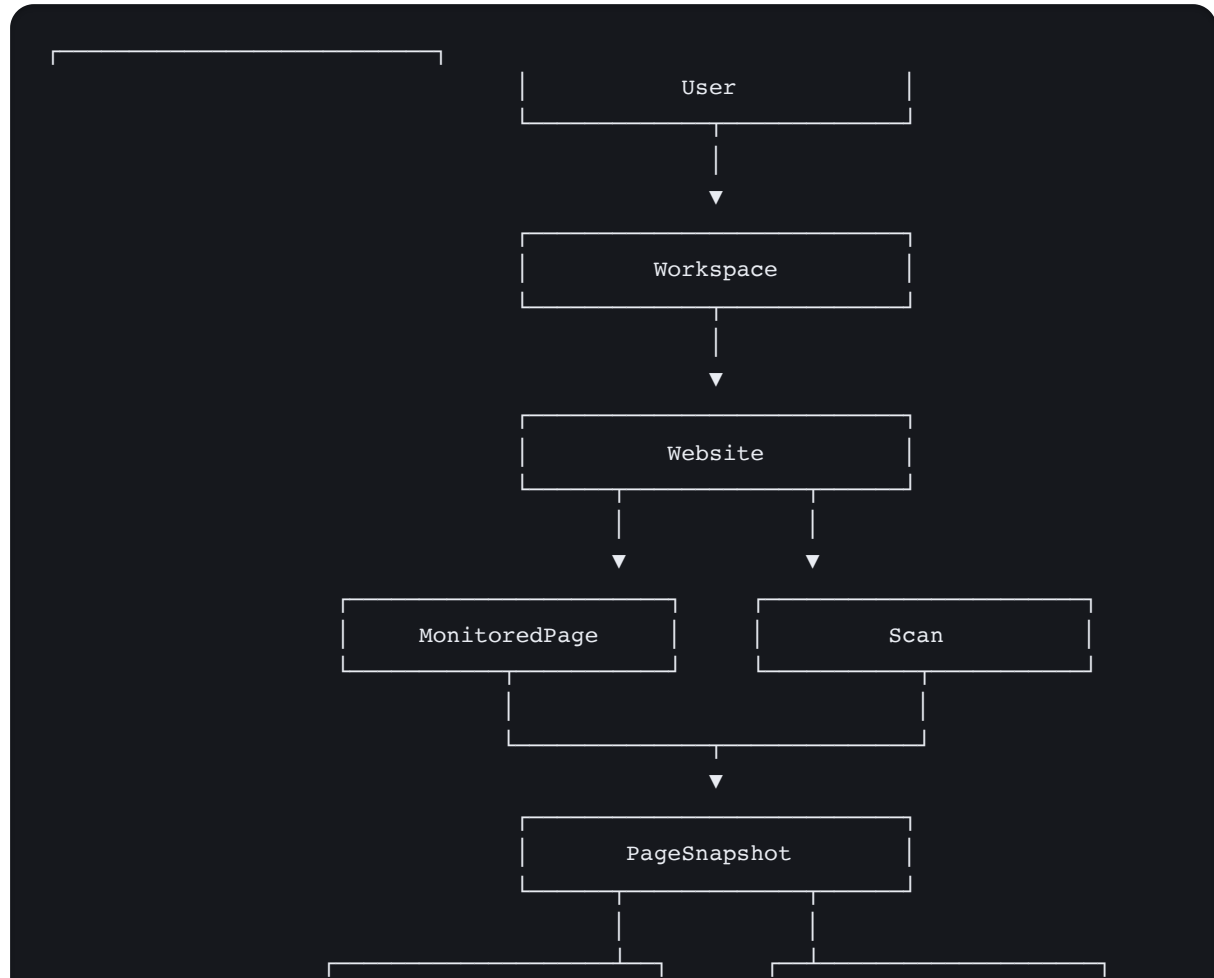
```

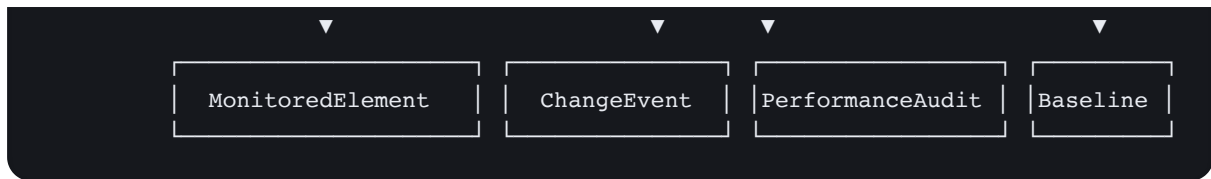
├── apps/
│   ├── web/           # Next.js 15 App Router (Dashboard, Marketing, APIs, SSR)
│   ├── worker/        # Node.js Scan Execution & Cron Sweeper Worker Process
│   └── mobile/         # Expo React Native App (iOS & Android)
├── packages/
│   ├── database/       # Prisma Schema, Migrations & Database Client
│   ├── scanner/        # Playwright Browser Pool, Extraction Engine, Storage &
│   ├── comparison-engine/ # Deterministic Snapshot & Visual Comparison Engines
│   ├── severity-engine/ # Rule-Based Severity Matrix Engine
│   ├── email/          # Resend & React Email Templates
│   └── shared/         # SSRF Pipeline, URL Normalizer, Limits, Channels & Types
├── docs/              # Master Specifications, Design System, Architecture & C
├── netlify.toml       # Netlify Deployment Configuration for Web App
└── pnpm-workspace.yaml # Monorepo Workspace Configuration

```

5. Database Schema & Data Model Analysis

The database schema (`packages/database/prisma/schema.prisma`) defines 22 models and 12 ENUM types structured across 7 functional domain areas:





5.1 Domain Model Breakdown

A. Authentication & Workspaces

- **User** : System account containing name, email, avatar image, two-factor auth status, and relation links to owned workspaces, memberships, approved baselines, and change notes.
- **TwoFactor** : Better Auth TOTP plugin storage holding encrypted secret keys, backup codes, and lockout counters.
- **Session** & **Account** : Managed session tokens, expiration dates, IP addresses, OAuth access tokens, and passwords.
- **Workspace** : Multi-tenant isolation container owning websites, team members, notification channels, billing subscriptions, and add-on capacity packs.
- **WorkspaceMember** : Maps users to workspaces with RBAC roles (**OWNER** , **ADMIN** , **MEMBER** , **VIEWER**).
- **WorkspaceInvite** : Cryptographically secure 24-byte invite tokens with 7-day expiration for workspace onboarding.

B. Website & Page Management

- **Website** : Core entity representing a monitored site. Stores **normalizedUrl** , **status** (**PENDING** , **DISCOVERING** , **BASELINING** , **ACTIVE** , **PAUSED** , **ERROR**), **scanFrequency** (**DAILY** , **WEEKLY**), stabilization rules (**ignoredSelectors** , **screenshotMasks**), organizational **tags** , and public status page/badge tokens (**publicToken**).
- **MonitoredPage** : Specific target URLs under a website selected for snapshotting and baseline comparison.

C. Scans & Capture Artifacts

- **Scan** : Represents one execution run (**triggerType** : **BASELINE** , **SCHEDULED** , **MANUAL**). Tracks progress (**pagesRequested** , **pagesScanned** , **pagesFailed**), detected change count, and highest severity level.
- **PageSnapshot** : Comprehensive snapshot of a page. Stores HTTP status, response time (ms), hashes (**htmlHash** , **domHash** , **textHash** , **screenshotHash** , **structuredDataHash**), S3 screenshot storage key, SEO metadata (title, meta description, canonical URL, robots meta, H1 array), page weight (bytes), and request count.
- **PageLink** : Individual links extracted during scan, classified as **INTERNAL** or **EXTERNAL** with status codes.
- **PageScript** : External JavaScript assets extracted during scan, tagged with domain, service identification, and **isThirdParty** flag.

- **SiteMetaSnapshot** : Site-level tracking for `robots.txt` and `sitemap.xml` content body, HTTP status, URL counts, and hashes.

D. Conversion & Regression Detection

- **MonitoredElement** : User-defined business-critical conversion targets (e.g., CTA buttons, pricing forms). Stores CSS selector, target expectations (`expectedExistence` , `expectedVisibility` , `expectedText` , `expectedHref`), and importance (`NORMAL` , `IMPORTANT` , `CRITICAL`).
- **MonitoredElementResult** : Per-snapshot capture of observed element state (`exists` , `visible` , `text` , `href`) to enable historical comparison.
- **Baseline** : Active approved version reference (`status` : `ACTIVE` , `SUPERSEDED`) pairing a `MonitoredPage` with an approved `PageSnapshot` .
- **ChangeEvent** : Structured output of comparison engines. Stores category (`AVAILABILITY` , `VISUAL` , `SEO` , `CONTENT` , `LINKS` , `SCRIPT` , `PERFORMANCE` , `CONVERSION`), `changeType` , `severity` , before/after values, and review status (`NEW` , `REVIEWED` , `APPROVED` , `RESOLVED` , `IGNORED`).
- **ChangeNote** : Discussion thread on change events for team collaboration.

E. Health & Performance Probes

- **PerformanceAudit** : Lighthouse audit results for a site URL. Stores 0–100 category scores (Performance, Accessibility, Best Practices, SEO) and Core Web Vitals (`lcpMs` , `fcpMs` , `tbtMs` , `ttiMs` , `speedIndexMs` , `cls`).
- **HealthCheck** : Lightweight GET probe of website homepages executed every 5 minutes by the worker process. Tracks response time, HTTP status, and SSL certificate expiration (`sslValidTo`).
- **HealthIncident** : Incident tracker for `DOWN` (opened after 2 consecutive failed probes) or `SSL_EXPIRING` (certificate expires within 14 days). Rate-limits notifications to once per 24h.

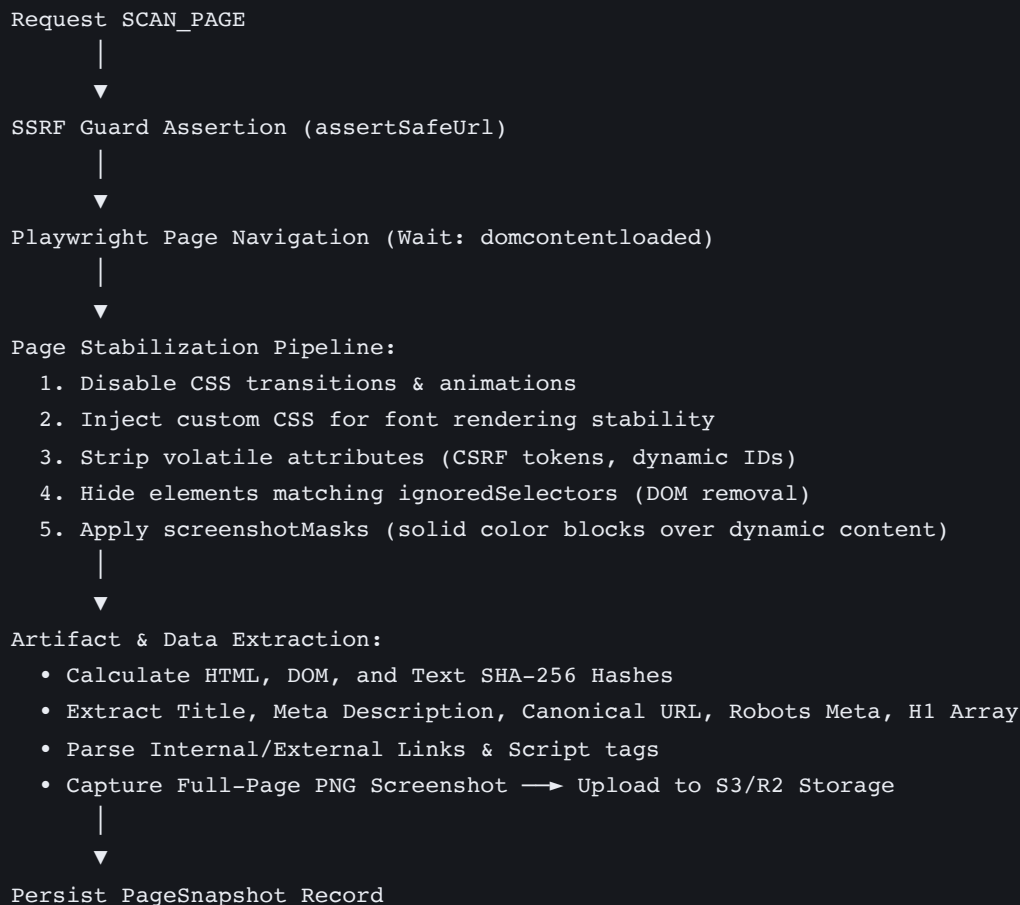
F. Notifications & Subscriptions

- **NotificationChannel** : Delivery channel setup per workspace (`EMAIL` , `SLACK` , `WEBHOOK` , `DISCORD` , `MICROSOFT_TEAMS`) with JSON threshold filters.
- **Notification** : Audit log of dispatched alert messages.
- **Subscription** : Workspace entitlement source of truth driven by Dodo Payments (`planId` : `free` / `pro` , `status` , `dodoSubscriptionId`).
- **CheckoutIntent** : Cryptographically signed checkout tokens preventing client-side metadata tampering.
- **WebsiteAddon** : Additional website capacity packs (\$6/mo for +30 sites each) allowing flexible scaling.

6. Deep Engine Implementations & Algorithms

6.1 Scanner Engine (`packages/scanner`)

The scanner engine orchestrates Playwright headless Chromium instances to render complex web pages with high determinism.



6.2 Comparison Engine (`packages/comparison-engine`)

The comparison engine performs deterministic diffing between a new `PageSnapshot` and its active `Baseline` :

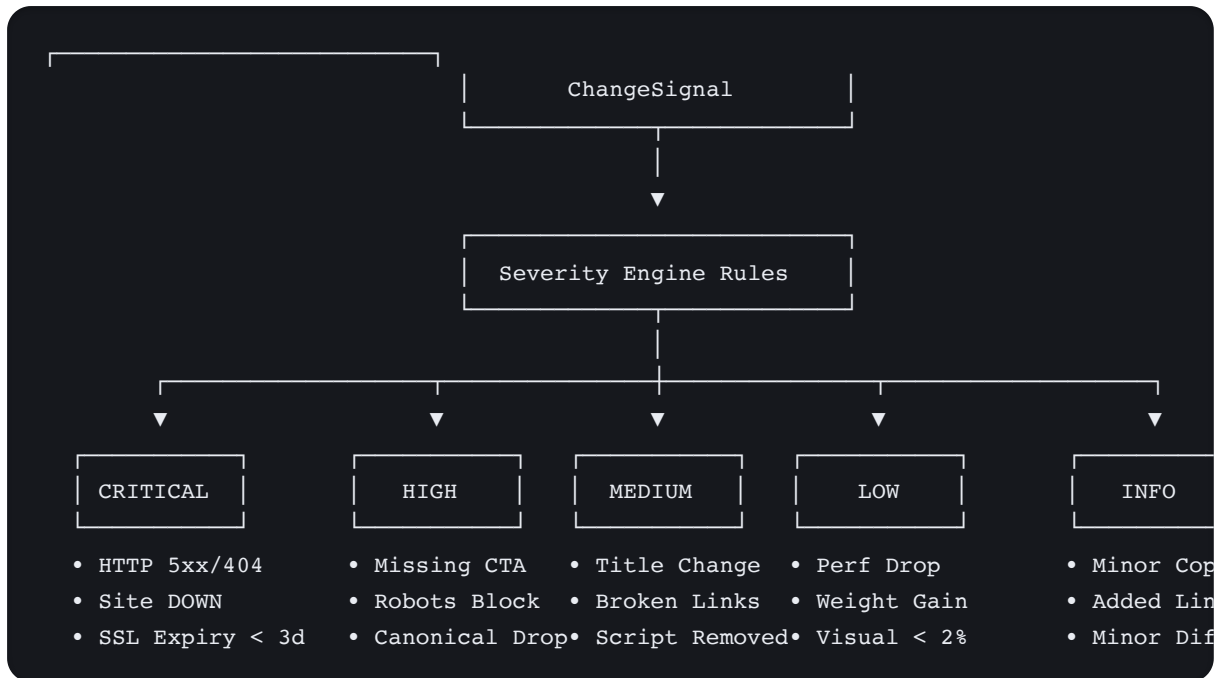
- Availability Diff:** Checks HTTP status code changes (e.g., `200 OK` → `500 Error` or `404 Not Found`).
- SEO Metadata Diff:** Diffing title strings, meta descriptions, canonical URLs, robots meta (`noindex` / `nofollow` drops), and H1 tag lists.
- Visual Diff (`visual.ts`):** Uses `pixelmatch` to compare PNG screenshots byte-for-byte. Generates a visual difference percentage (`visualDifferencePercentage`). If diff > threshold (e.g., 0.1%), marks a visual change.
- DOM & Content Diff:** Compares clean DOM SHA-256 hashes and visible text hashes to isolate structural edits from cosmetic copy changes.
- Link & Script Diff (`links.ts`):** Identifies removed internal links, added third-party tracking scripts, or missing core JS bundles.
- Site Metadata Diff (`site-meta.ts`):** Compares `robots.txt` disallow rules and

`sitemap.xml` URL count drops across site scans.

7. **Monitored Element Assertion:** Compares observed conversion element state (`MonitoredElementResult`) against expected text, visibility, and href values.

6.3 Severity Engine (`packages/severity-engine`)

All severity scoring is centralized inside `packages/severity-engine/src/index.ts`. No severity calculations occur in the frontend or worker loops.



6.4 SSRF Security Pipeline (`packages/shared/src/ssrf.ts`)

Because MyKavo fetches user-supplied URLs, outbound HTTP requests are treated as untrusted. The SSRF guard enforces a strict 12-step validation pipeline (`safeFetch`):

1. Parse & validate URL format.
2. Restrict protocols strictly to `http:` and `https:`.
3. Reject embedded user credentials (`user:pass@host`).
4. Block internal hostnames (`localhost`, `.local`, `.internal`).
5. Perform dual IPv4 and IPv6 DNS lookup (`lookup()`).
6. Reject loopback (`127.0.0.0/8`, `::1`).
7. Reject private RFC 1918 ranges (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`).
8. Reject link-local & cloud metadata IPs (`169.254.169.254`, `fd00:ec2::254`).
9. Reject carrier-grade NAT, broadcast, and multicast addresses.
10. Disable automatic redirect following; manually validate each redirect hop against steps 1–9 (max 5 hops).
11. Enforce strict connection timeout (10 seconds).
12. Cap response body payload size (max 2 MB for free tools, stream-cut).

7. Background Worker & Automated Sweeps (`apps/worker`)

The worker app (`apps/worker/src/index.ts`) operates as a resilient background process running recurring jobs:

`apps/worker` Execution Loops

1. Scan Scheduler (Every 1 min)	→ Queries websites due for scan → enqueues S
2. Health Sweep (Every 5 mins)	→ Lightweight HTTP GET & SSL probe across AC
3. Audit Sweep (Every 6 hours)	→ Executes Lighthouse performance audits
4. Billing Sweep (Every 12 hours)	→ Syncs Dodo Payments subscription states &
5. Retention Sweep (Daily 03:00)	→ Purges expired snapshots & scans past plan

- **False-Positive Health Guard:** A site is marked `DOWN` only after **two consecutive failed checks**, eliminating single-network blip alerts.
- **Baseline-Safe Retention:** The retention sweep never purges snapshots that serve as an **active baseline**, ensuring historical integrity.

8. Web App, API Routes & Mobile Infrastructure

8.1 Next.js Web App Structure (`apps/web`)

- **Marketing Pages* (`/`, `/pricing`, `/about`, `/blog`, `/guides`, `/tools/`): **SEO-optimized public pages rendering product features, free tools, and pricing plans.**
- *Dashboard Routes* (`/dashboard/`):

`/dashboard` : Overview cards, health indicators, recent change stream. `/dashboard/websites` : Website listing, add website wizard, selector settings. `/dashboard/changes` : Global change triage list, category/severity filters. `/dashboard/scans` : Historical scan log and detail inspection modal. `/dashboard/settings` : Workspace settings, team invite management, notification channels, Dodo billing portal.

- **Public Status Pages** (`/status/[token]`): Whitelabeled uptime status pages for client sharing.
- **Uptime Badges** (`/api/badge/[token]`): Dynamic SVG badge endpoints for website footers (`Uptime 99.9%`).

8.2 Mobile Client (`apps/mobile`)

Built using **Expo** and **Expo Router**, the mobile app provides on-the-go monitoring for agency leads:

- **Tabs:** Home Overview, Websites List, Scans History, Change Triage, Settings.
- **Biometric Auth:** Secure login via Face ID / Touch ID using `SecureStore` .
- **Push Notifications:** Instant alert delivery for `CRITICAL` site outages or CTA failures.

9. Environment Variables & Configuration

Below is the exhaustive matrix of required environment variables (`.env`):

```
# Database Connection (PostgreSQL)
```

```
DATABASE_URL="postgresql://user:password@host:5432/mykavo?sslmode=require"
```

Auth Configuration (Better Auth)

```
BETTER_AUTH_SECRET="your-super-secret-random-32-byte-string"
```

```
BETTER_AUTH_URL="http://localhost:3000" # Production: https://mykavo.com
```

S3 / Cloudflare R2 Object Storage (Screenshots)

```
S3_ENDPOINT="https://<account-id>.r2.cloudflarestorage.com"
```

```
S3_BUCKET="mykavo-screenshots"
```

```
S3_REGION="auto"
```

```
S3_ACCESS_KEY_ID="your-s3-access-key-id"
```

```
S3_SECRET_ACCESS_KEY="your-s3-secret-access-key"
```

Email Delivery (Resend)

```
RESEND_API_KEY="re_123456789"
```

```
EMAIL_FROM="MyKavo Alerts <alerts@mykavo.com>"
```

Dodo Payments (Billing & Subscriptions)

```
DODO_PAYMENTS_API_KEY="dodo_live_xxxxxxx"
```

```
DODO_PAYMENTS_WEBHOOK_SECRET="whsec_xxxxxxx"
```

```
DODO_PAYMENTS_ENVIRONMENT="live" # or "test"
```

Worker & System Tuning

```
WORKER_CONCURRENCY=5
```

```
PLAYWRIGHT_MAX_BROWSERS=2
```

10. Developer Guide: Rules for Continuing Development Safely

To ensure seamless continuation of the project without introducing regressions, breaking deployments, or corrupting data, all developers and AI agents (Antigravity / Claude) MUST follow these mandatory rules:

Rule 1: Never Break the Pre-Flight Verification Loop

Before proposing or committing any code changes, always execute the full workspace check command:

```
pnpm --filter web typecheck && pnpm lint && pnpm test
```

All tests must pass cleanly before any code is approved.

Rule 2: Database Schema Changes Require Prisma Migrations

- Never mutate `schema.prisma` without running a proper migration or type generation.
- Update `@mykavo/database` using:

```
pnpm --filter database db:push
```

- Ensure `binaryTargets = ["native", "rhel-openssl-3.0.x"]` is preserved in `schema.prisma` so serverless functions on Netlify do not crash with missing OpenSSL engine errors.

Rule 3: Maintain Centralized Severity & SSRF Guards

- Never hardcode severity ratings outside `packages/severity-engine/src/index.ts`.
- Never perform outbound HTTP fetches on user-influenced URLs without routing through `safeFetch` from `@mykavo/shared/ssrf`.

Rule 4: Preserve Baseline Data Integrity

- Never execute `DELETE` queries on `PageSnapshot` rows without ensuring `status !== 'ACTIVE'` on associated `Baseline` records.
- Baseline snapshots are immutable historical anchor points.

Rule 5: Design Token Compliance

- Always use the CSS custom variables (`--fx-`) defined in `globals.css`.
- Never use raw arbitrary hex colors (e.g., `#3556f4`) directly in component markup—always use design system tokens (`bg-primary` , `text-ink` , `border-line`).
- Maintain color contrast WCAG AA compliance verified by `theme-contrast.test.ts`.

11. Project Status & Roadmap

<i>[Phase 0] Validation Assets & Landing Page</i>	✓ Complete
<i>[Phase 1] Core Monorepo & Auth (Better Auth)</i>	✓ Complete
<i>[Phase 2] Website Management & URL Discovery</i>	✓ Complete
<i>[Phase 3] Scanning Engine & Playwright Browser Pool</i>	✓ Complete
<i>[Phase 4] Baselines & Versioning</i>	✓ Complete
<i>[Phase 5] Comparison & Severity Engines</i>	✓ Complete
<i>[Phase 6] Change Triage Interface & Diff Modal</i>	✓ Complete
<i>[Phase 7] Scheduling & Email Notifications</i>	✓ Complete
<i>[Phase 8] Dodo Payments Billing & Add-on Packs</i>	✓ Complete
<i>[Phase 9] Conversion Element CTA Monitoring</i>	✓ Complete
<i>[Phase 10] Production Hardening & Retention Cleanups</i>	✓ Complete
<i>[Phase 11] SEO Growth Engine & Public Tool Expansion</i>	◀ NEXT IN LINE

MyKavo Master Case Study & Technical Architecture Document — Monorepo Build Version 1.0.0*